

Loss Functions for Model Training

Prepared by Ibrahim Sammour (Info 407 Course)

Definition

To train a model, we tweak the weights and biases to improve its accuracy and confidence in output. The **loss function (cost function)** is an algorithm that quantifies how wrong a model is. Since the loss is the error of the model, we want to minimize it to reach 0 ideally.

A small loss value means we are more confident about our prediction. A small example would be a neural network with a softmax activation in the output layer. Suppose at two different stages of training we have the following outputs for the same input:

- Output 1: $[0.45, 0.15, 0.4]$
- Output 2: $[0.9, 0.03, 0.07]$

In this case, output 2 is more accurate, thus its loss (if label one is the true prediction) is less than the loss obtained with output 1.

Loss Functions

Classification

Categorical Cross-Entropy Loss (CCE)

It is used to compare the "ground-truth" probability (y) and the predicted probability distribution ($\hat{y}$). It is usually used with the softmax activation function in the output layer.

The formula is

$$L_i = - \sum_{j=1}^C y_{i,j} \log(\hat{y}_{i,j})$$

Where:

- L_i is the loss value of the i^{th} sample
- j is the index of the class
- C is the number of classes
- y is the ground truth/target value
- $\hat{y}$ is the predicted value

Example

Consider the example from the definition. Softmax outputs:

- Output 1: $[0.45, 0.15, 0.4]$
- Output 2: $[0.9, 0.03, 0.07]$
- Ground Truth/ Target: $[1, 0, 0]$

Sample 1:

$$L_i = -(1 \log(0.45) + 0 \log(0.15) + 0 \log(0.4)) = -\log(0.45) = 0.3467$$

Sample 2:

$$L_i = -(1 \log(0.9) + 0 \log(0.03) + 0 \log(0.07)) = -\log(0.9) = 0.04575$$

This explains why we use log instead of doing $(y_j - \hat{y}_j)$. The difference in loss is not linear and is more significant.

A few notes:

- If the model assigns a very small probability to the correct class (say 0.001), the log will produce a large loss (approx. 6.9), heavily penalizing the model.
- This loss is minimized when the model assigns high probability to the correct class. Since $\log(1) = 0$, a perfect prediction leads to a zero loss. The farther the predicted probability from 1 for the true class, the higher the loss/penalty.
- Categorical Cross-Entropy is used when we have **one-hot encoded labels (targets)**, where for each sample we save a vector of size = number of classes. Obviously, this is inefficient.

Sparse Categorical Cross Entropy (SCCE)

Unlike CCE, SCCE does not use one-hot encoded labels, instead, it only uses the index of the true label. This is far more memory efficient than CCE, which stores one-hot encoded labels in memory.

Our previous example becomes:

- Output 1: $[0.45, 0.15, 0.4]$
- Output 2: $[0.9, 0.03, 0.07]$
- Ground Truth/ Target: $[0]$ (it was $[1, 0, 0]$ previously).

CCE vs SCCE

We usually prefer to use CCE over SCCE in the following cases:

- Having a small number of classes, so we have to store a small vector for each sample, for example, $[0.45, 0.15, 0.4]$.
- When we are interested in making the model less confident about its decision to make it generalize better and avoid overfitting. One example is **Label Smoothing** (explained shortly).

We usually prefer to use SCCE over CCE in the following cases:

- When the number of classes is large, e.g., 2000 classes, thus we have to store a vector of size 2000 per sample.
- When the classes are mutually exclusive: we don't care about close enough predictions.

Note: Label Smoothing

$$\tilde{y}_j = \begin{cases} 1 - \epsilon & \text{if class } j \text{ is the true class} \\ \frac{\epsilon}{C-1} & \text{if class } j \text{ is not the true class} \end{cases}$$

We adjust the prediction slightly to make the model less confident. A small adjustment ϵ is made as per the equation where C is the number of classes.

Usually used in NLP and in the case of noisy data:

- Helps reduce overfitting and prevent the model from becoming too confident in rare words.
- When the data contains some noisy or mislabeled samples, label smoothing can help the model generalize better.

A one-hot encoded target becomes: $[1, 0, 0] \Rightarrow [0.95, 0.05, 0.05]$

Note: Imbalanced Datasets

One way to deal with imbalanced datasets is to use weighted loss functions where we assign each class importance (weight). When we have datasets of diseases, for example (rare diseases have fewer samples), weighted loss functions help mitigate this bias by assigning more weight to underrepresented classes. (We have two types: class weight and sample weight).

Binary Cross-Entropy Loss (BCE)

BCE is a variant of CCE where we have a two-class problem. The formula of BCE is:

$$L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

Where y is the true label.

When the true label is 1 (positive class): The loss is calculated as $-\log(\hat{y})$. If $\hat{y}$ is close to 1 (the model is confident and correct), the loss will be very small. If $\hat{y}$ is small (close to 0), the loss will be large (indicating a poor prediction).

When the true label is 0 (negative class): The loss is calculated as $-\log(1 - \hat{y})$. If $\hat{y}$ is close to 0 (the model correctly predicts class 0), the loss will be small. If $\hat{y}$ is close to 1 (incorrectly predicting the positive class), the loss will be large.

Example 1: True label is 1 (positive class)

Suppose the true label y is 1, and the predicted probability $\hat{y}$ is 0.8.

$$L = -(1 \log(0.8) + (1 - 1) \log(1 - 0.8)) = -\log(0.8) = 0.2231$$

In this case, the loss is relatively small because the model is confident (0.8) about its prediction being correct.

Example 2: True label is 0 (negative class)

Suppose the true label y is 0, and the predicted probability $\hat{y}$ is 0.3.

$$L = -(0 \log(0.7) + (1 - 0) \log(1 - 0.7)) = -\log(0.7) = 0.3567$$

Here, the model predicted a probability of 0.3 for class 1, which is fairly confident about predicting class 0. Thus, the loss is relatively small.

Example 3: True label is 0, but predicted probability for class 1 is 0.9

$$L = -(0 \log(0.9) + (1 - 0) \log(1 - 0.9)) = -\log(0.1) = 2.3026$$

In this case, the loss is large because the model confidently predicted the positive class (0.9 for class 1) when the true class was 0, signaling a significant error.

Regression

Mean Squared Error (MSE)

MSE is usually used in regression tasks. We square the difference between the predicted and the true value of each output in the sample and average those square differences.

The general equation is:

$$L_i = \frac{1}{J} \sum_{j=1}^J (y_{i,j} - \hat{y}_{i,j})^2$$

Where:

- i : is the current sample
- j : is the current output
- J : is the number of outputs

The idea of MSE is to penalize harshly as we move away from the target. MSE performs badly in case of outliers as they dominate the loss (very large error values).

Practically, we prefer to use MAE.

Mean Absolute Error (MAE)

$$L_i = \frac{1}{J} \sum_{j=1}^J |y_{i,j} - \hat{y}_{i,j}|$$

Here we use absolute instead of squaring, so outliers will not have much greater impact.

Huber Loss

It combines MSE and MAE:

- When the error between the target and the predicted values is small $\Rightarrow$ uses MSE.
- When the error is large $\Rightarrow$ uses MAE.